

Project P12 - Serverless Event-Driven Workflow

Student: Betea Adrian-Vlad

Course: DevOps

1. Introduction

In this project I implemented a Serverless Event-Driven Workflow using AWS functions (Lambda), Amazon SQS (Simple Queue Service), Amazon DynamoDB (database), Amazon API Gateway and a simple html frontend simulating a Virtual Campus where students can see assigned courses and enroll to new courses.

The technologies used for each component are:

- Fronted UI: A simple HTML/JS page with a built-in timer (`performance.now()`) to measure execution time and observe the cold start phenomenon.
- Amazon API Gateway: Handles the HTTP requests coming from the browser.
- AWS Lambda1 (Producer/API): A Python 3.14 function that receives HTTP requests. It reads data directly from the database or sends enrollment requests to a queue.
- Amazon SQS (Simple Queue Service): A message queue that acts as a buffer for the enrollment requests.
- AWS Lambda2 (Worker): A background Python function triggered automatically by SQS to save data into the database.
- Amazon DynamoDB: A NoSQL database to store the student profiles and their courses.

The main DevOps concepts of this serverless architecture are: cold/warm start phenomenons, zero idle cost/pay-as-you-go, security and IAM (identity and access management).

2. System Architecture and Workflow

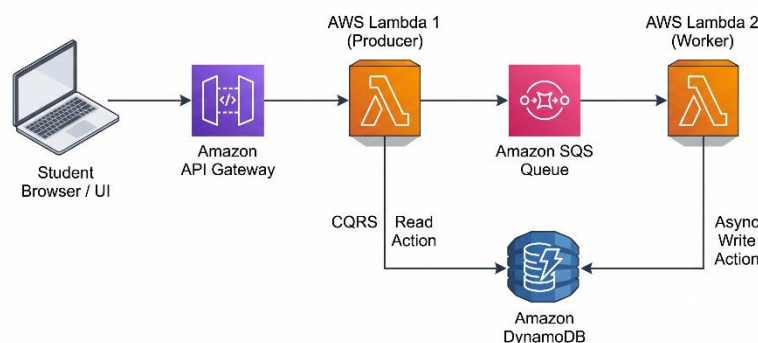


Figure 1: System Architecture

In Figure 1 we can see the serverless system architecture of the Virtual Campus application. The flow begins with the Frontend UI (on the left), which communicates with the cloud backend through HTTP requests. These requests are intercepted by the Amazon API Gateway, which acts as the front door for the system and triggers the first compute service: AWS Lambda 1 (The Producer).

From Lambda 1, the architecture splits into two paths based on the type of action requested by the student. If the student wants to view their courses, Lambda 1 connects directly to the Amazon DynamoDB database. Otherwise, if the student wants to enroll in a new course (a write action), Lambda 1 sends the data to an Amazon SQS Queue. The queue then automatically triggers AWS Lambda 2 (The Worker), which processes the requests in the background and updates the database.

This structure of the system makes the application highly scalable and prevents database crashes during high traffic (for example, when hundreds of students try to enroll at the same time). The workflow is strictly divided into separate paths: Reading data and Writing data.

A. Reading Workflow (Synchronous)

When a student logs into the Virtual Campus, the system needs to fetch their currently assigned courses.

1. The frontend sends an HTTP GET request to the Amazon API Gateway.
2. The API Gateway triggers Lambda 1.
3. Lambda 1 connects directly to DynamoDB, retrieves the student's courses, and sends the data back to the browser.

B. Writing Workflow (Asynchronous / Event-Driven)

When a student selects a course and clicks "Enroll", the system does not write directly to the database. Instead, it uses an event-driven background process:

1. The frontend sends an HTTP POST request to the API Gateway, which triggers Lambda 1.
2. Lambda 1 packages the enrollment requests into a JSON message and sends it to Amazon SQS Queue, it then immediately replies to the student with a success message.
3. The arrival of the new message in the SQS Queue acts as an Event. This event automatically triggers Lambda 2 (Worker) in the background.
4. Lambda 2 reads the message from the queue and appends the new course to the student's record in the DynamoDB

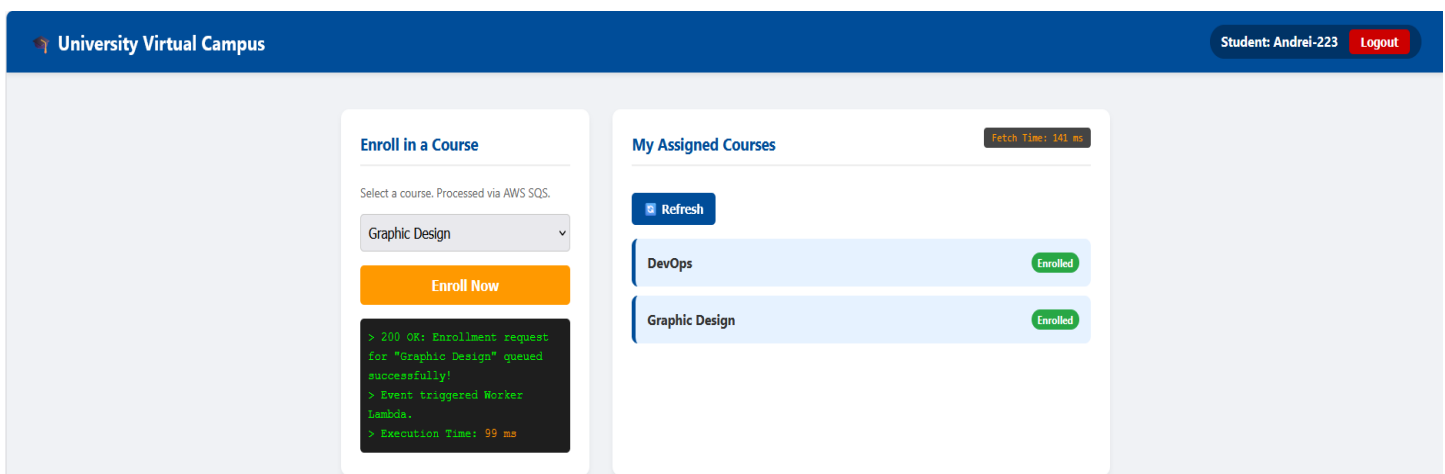


Figure 2: Virtual Campus Frontend

Students Autopreview [View table details](#)

▼ **Scan or query items** info

☒ Scan
 ☐ Query

Select a table or index: Table - Students
 Select attribute projection: All attributes

► **Filters - optional**

[Run](#)
[Reset](#)

✓ **Completed** · Items returned: 2 · Items scanned: 2 · Efficiency: 100% · RCUs consumed: 2

Table: Students - Items returned (2)
Actions
Create item

Scan started on May 16, 2026, 20:26:17

☐	id (String)	courses	LastUpdated
☐	Andrei-223	[{ "S" : "DevOps" }, { "S" : "Graphic Design" }]	2026-05-16 20:06:19
☐	Alex-123	[{ "S" : "DevOps" }, { "S" : "Introduction to Research" }, { "S" : "Cloud Foundations" }]	2026-05-16 15:33:19

Figure 3: The database records

As we can see in Figure 2, the fetch time for the assigned courses is available on top right of “My Assigned Courses” container, if we press on the “refresh button” we trigger the reading workflow, Amazon API Gateway -> Lambda1 (read from database) -> read from Amazon DynamoDB. And in the left container we have the “Enroll in a Course” container where the student can choose a course from a dropdown list and if presses on the Enroll Now button it triggers the writing workflow, Amazon API Gateway -> Lambda1 -> Simple Queue Service -> Lambda 2 (Worker, writes the course in database) -> stored in Amazon DynamoDB. In Figure 3 we can see the DynamoDB with the records.

3. DevOps Concepts and Challenges Solved

During the implementation of this serverless architecture, several core DevOps concepts and practical challenges were addressed:

1. The “Cold Start” Phenomenon

In a serverless environment, AWS does not keep the servers running when there is no traffic. If a Lambda function has not been used recently, its container is destroyed.

- **Cold Start:** when a new request arrives, AWS must provision a new container and load the Python environment. I measured this using a timer in the frontend. A Cold Start typically takes between 1000ms and 2000ms.
- **Warm Start:** If another request is made after, the container is already running and kept in memory. The execution time dropped to around 80ms to 160ms.

2. Cost Implications

A major advantage of this architecture is the financial model.

- **Zero Idle Cost:** Unlike traditional virtual machines (like AWS EC2) where you pay an hourly rate even if no students are using the portal, this serverless architecture costs \$0 when idle.

- Pay-As-You-Go: AWS bills Lambda based on the number of requests and the execution duration (in milliseconds). Because Lambda 1 simply drops a message in the SQS queue and closes quickly, the compute costs are kept to an absolute minimum.

3. Security and IAM (Identity and Access Management)

By default, AWS services operate on a “Zero-Trust” model and cannot communicate with each other. To make the workflow function securely, I had to create specific IAM Execution Roles:

- Lambda 1 was granted permission to write to SQS and read from DynamoDB.
- Lambda 2 was granted the `AWSLambdaSQSQueueExecutionRole` policy to be triggered by the queue, and permissions to write to DynamoDB.

4. Practical Challenges Solved

- CORS (Cross-Origin Resource Sharing): Because the local frontend HTML file communicates with an external AWS API domain, the browser blocked the requests for security reasons. I solved this by explicitly injecting the `Access-Control-Allow-Origin: *` headers into the Lambda Python response.
- Preventing Duplicate Enrollments and Race Conditions: critical issue discovered during testing was that if a student clicked "Enroll" multiple times rapidly, duplicate messages were sent to the queue, resulting in duplicate courses in the database. To fix this, I added a `ConditionExpression` directly to the DynamoDB update logic in Lambda 2. The database now acts as a lock, strictly rejecting the update if the course is already in the student's list. Additionally, I added client-side validation to disable the "Enroll" button during processing.